



infoceumbs

JSHint report

Created using Code Institute
JSHint

checkout/stripe_elements.js

JSHintInterface

Check Key

Filename: stripe_element

Mandatory if you a

URL: Enter URL

Options: ☒ ES6 ☒ ES5 ☐ Hash ☐ jQuery ☐ Relax ☒ Strict

Code:

JSHint Results for stripe_elements.js

No errors reported

Close

```
// Retrieve values from Django's json_script tags using JQuery selectors
const stripePublicKey = $('#id_stripe_public_key').text().slice(1, -1);
const clientSecret = $('#id_client_secret').text().slice(1, -1);
const planId = $('#id_plan_id').val();
const frequencyId = $('#id_frequency_id').val();

// Retrieve user full name and email from json_script tags
const userFullName =
JSON.parse(document.getElementById('id_user_full_name').textContent);
const userEmail =
JSON.parse(document.getElementById('id_user_email').textContent);

const stripe = Stripe(stripePublicKey);
const elements = stripe.elements();

// Define custom styling for the Stripe Elements to match Bootstrap/Crispy
Forms dark theme
const style = {
  base: {
    color: '#f0f0f0', // Bright text color for input (contrasts with
dark background)
    fontFamily: '"Helvetica Neue", Helvetica, sans-serif', //
Consistent font
    fontSmoothing: 'antialiased',
    fontSize: '16px',
    '::placeholder': {
      color: '#adb5bd' // Slightly muted placeholder text color
    },
    // CRITICAL: Adjusted padding for vertical centering.
    // (40px total height - 16px font-size) / 2 = 12px top/bottom
padding
padding: '16px 12px', // Top/bottom 16px, left/right 12px
backgroundColor: '#343a40', // Explicitly set background color
for the input field itself
    '-webkit-autofill': {
      color: '#f0f0f0', // Text color for autofilled fields
      backgroundColor: '#343a40', // Background for autofilled fields
    },
    // Removed explicit 'height' and 'lineHeight' from here,
    // relying on external CSS for overall container height and
padding for internal spacing.
  },
  invalid: {
    color: '#f0755a', // Error text color
    iconColor: '#f0755a' // Error icon color
  }
};

// Create the card element and hide the postal code
const card = elements.create('card', {
  style: style,
  hidePostalCode: true // This hides the ZIP/Postal Code field
});

// Mount the card element to the div with id="card-element"
card.mount('#card-element');

// Handle real-time validation errors on the card element
card.addEventListener('change', function (event) {
  const errorDiv = document.getElementById('card-errors');
  if (event.error) {
    $errorDiv.text(event.error.message);
  } else {
    $errorDiv.text('');
  }
});

// Handle form submission
const form = document.getElementById('payment-form');

if (form) {
  form.addEventListener('submit', function(ev) {
    ev.preventDefault();
    card.update({ 'disabled': true });
    $submitButton.attr('disabled', true);
    $loadingOverlay.fadeIn(100);

    const cardToken = $('#input[name="cardtoken"]').val();
    const postData = {
      'cardtoken': cardToken,
      'client_secret': clientSecret,
      'plan_id': planId,
      'frequency_id': frequencyId,
    };
    const url = '/checkout/cache_checkout_data/';

    $.post(url, postData).done(function () {
      stripe.confirmCardPayment(clientSecret, {
        payment_method: {
          card: card,
          billing_details: {
            name: userFullName,
            email: userEmail,
          }
        }
      }).then(function(result) {
        if (result.error) {
          const errorDiv = document.getElementById('card-
errors');
          $errorDiv.text(result.error.message);
          card.update({ 'disabled': false });
          $submitButton.attr('disabled', false);
          $loadingOverlay.fadeOut(100);
        } else {
          if (result.paymentIntent.status === 'succeeded') {
            window.location.replace('/checkout/success/' +
result.paymentIntent.id + '/');
          }
        }
      });
    }).fail(function () {
      location.reload();
    });
  });
}
```

Run Checks

feedback/comments.js

JSHintInterface

Check Key

Filename: comments.js

URL: Enter URL

Options: ☒ ES5 ☒ ES6 ☐ JSHint ☐ Relax

Code:

JSHint Results for comments.js

No errors reported!

Close

```
document.addEventListener("DOMContentLoaded", function () {
  const commentForm = document.querySelector("#comment-form form");
  const commentsSection = document.getElementById("comments-section");

  if (commentForm) {
    commentForm.addEventListener("submit", function (e) {
      e.preventDefault();

      const formData = new FormData(commentForm);
      const actionUrl = commentForm.getAttribute("action");

      fetch(actionUrl, {
        method: "POST",
        body: formData,
        headers: {
          "X-Requested-With": "XMLHttpRequest"
        }
      })
        .then(response => {
          if (!response.ok) throw new Error("Failed to post comment.");
          return response.text();
        })
        .then(html => {
          commentsSection.innerHTML = html;
          commentForm.reset();
        })
        .catch(err => {
          console.error(err);
          alert("Error posting comment. Try again.");
        });
    });

    commentsSection.addEventListener("click", function (e) {
      const target = e.target;

      if (target.classList.contains("edit-comment")) {
        e.preventDefault();
        const commentId = target.dataset.id;

        fetch(`/feedback/edit/${commentId}/`, {
          headers: {
            "X-Requested-With": "XMLHttpRequest"
          }
        })
          .then(res => res.text())
          .then(html => {
            const container =
              document.getElementById("comment-" + commentId);
            if (container) {
              container.innerHTML = html;
            }
          });
      }

      if (target.classList.contains("delete-comment")) {
        e.preventDefault();
        const commentId = target.dataset.id;

        if (confirm("Delete this comment?")) {
          fetch(`/feedback/delete/${commentId}/`, {
            method: "POST",
            headers: {
              "X-Requested-With": "XMLHttpRequest",
              "X-CSRFToken": getCSRFToken()
            }
          })
            .then(res => res.json())
            .then(data => {
              if (data.success) {
                const el =
                  document.getElementById("comment-" + commentId);
                if (el) el.remove();
              } else {
                alert("Failed to delete comment.");
              }
            });
        }
      }
    });

    commentsSection.addEventListener("submit", function (e) {
      if (e.target.classList.contains("edit-comment-form")) {
        e.preventDefault();
        const form = e.target;
        const commentId = form.dataset.id;
        const formData = new FormData(form);

        fetch(`/feedback/edit/${commentId}/`, {
          method: "POST",
          body: formData,
          headers: {
            "X-Requested-With": "XMLHttpRequest",
            "X-CSRFToken": getCSRFToken()
          }
        })
          .then(res => res.json())
          .then(data => {
            if (data.success) {
              const el =
                document.getElementById("comment-" + commentId);
              if (el) {
                el.innerHTML = `
                <p>
                  <strong>You/strong> - just now</p>
                  <p class="comment-content">${data.content}
                </p>
                <div class="comment-actions">
                  <a href="#" class="edit-comment" data-id="${commentId}>Edit</a> |
                  <a href="#" class="delete-comment" data-id="${commentId}>Delete</a>
                </div>
                `;
              } else {
                alert("Failed to update comment.");
              }
            }
          })
          .catch(err => {
            console.error(err);
            alert("Error saving comment. Try again.");
          });
      }
    });

    function getCSRFToken() {
      const csrfOutput =
        document.querySelector("input[name=csrfmiddlewaretoken]");
      return csrfOutput ? csrfOutput.value : "";
    }
  });
});
```

Run JSHint

feedback/saved.js

JSHintinterface

Check Key

Filename: saved.js

Mandatory if you a

URL: Enter URL

Enter a URL or paste a GitHub repository

Options: ☒ ES6 ☒ ES8 ☐ Harsh ☐ JQuery ☐ Relax ☐ Strict

Code:

JSHint Results for saved.js

No errors reported!

Close

```
document.addEventListener("DOMContentLoaded", function () {
  const saveButton = document.getElementById("save-btn");

  if (saveButton) {
    saveButton.addEventListener("click", function () {
      const crumbId = this.dataset.id;

      if (!crumbId) {
        console.error("Missing crumb ID");
        return;
      }

      fetch(`/feedback/crumb/save/${crumbId}/`, {
        method: "POST",
        headers: {
          "X-CSRFToken": getCSRFToken(),
          "X-Requested-With": "XMLHttpRequest"
        }
      })
        .then(response => response.json())
        .then(data => {
          if (data.saved) {
            this.innerHTML = '<i class="fa-solid fa-bookmark">
</i> Saved';
          } else {
            this.innerHTML = '<i class="fa-regular fa-bookmark">
</i> Save';
          }
        })
        .catch(err => {
          console.error("Error saving crumb:", err);
        });
    });
  }

  function getCSRFToken() {
    return
document.querySelector('[name=csrfmiddlewaretoken]').value;
  }
});
```

Run Checks